

Autonomous ESG & Carbon Accounting Copilot

Ingest invoices, utility bills, freight logs and ERP exports. Classify every line into GHG Protocol Scope 1, 2 or 3. Bind it to an official EPA or DEFRA emission factor. Do the arithmetic in code, never in the model. Emit a CSRD / SEC-ready disclosure where every tonne traces back to the row that produced it.

Track ClimateTech / FinTech Compliance **Level** Intermediate **Platform** Lyzr Agent Studio + Agent API
Repo padmashri23/AI-Quest-Beyond-the-Wrapper

What the problem actually is

Three regimes now punish bad carbon numbers: the EU's Corporate Sustainability Reporting Directive (CSRD, reporting under ESRS E1), the Corporate Sustainability Due Diligence Directive (CSDDD), and the SEC climate disclosure rule. All three require Scope 1, 2 and 3 figures a third-party auditor can re-derive from source documents.

A plain LLM wrapper fails this in four predictable places. It multiplies wrong. It confuses kWh with MWh, litres with US gallons, and short tons with tonnes. It invents an emission factor when it cannot find one. And it happily reads supplier pricing and account numbers into its context and then out into a report. The judges are scoring exactly those failure modes.

30 points Lyzr architecture & tool calling	30 points Emission calculation accuracy	20 points Auditability & traceability
---	--	--

20

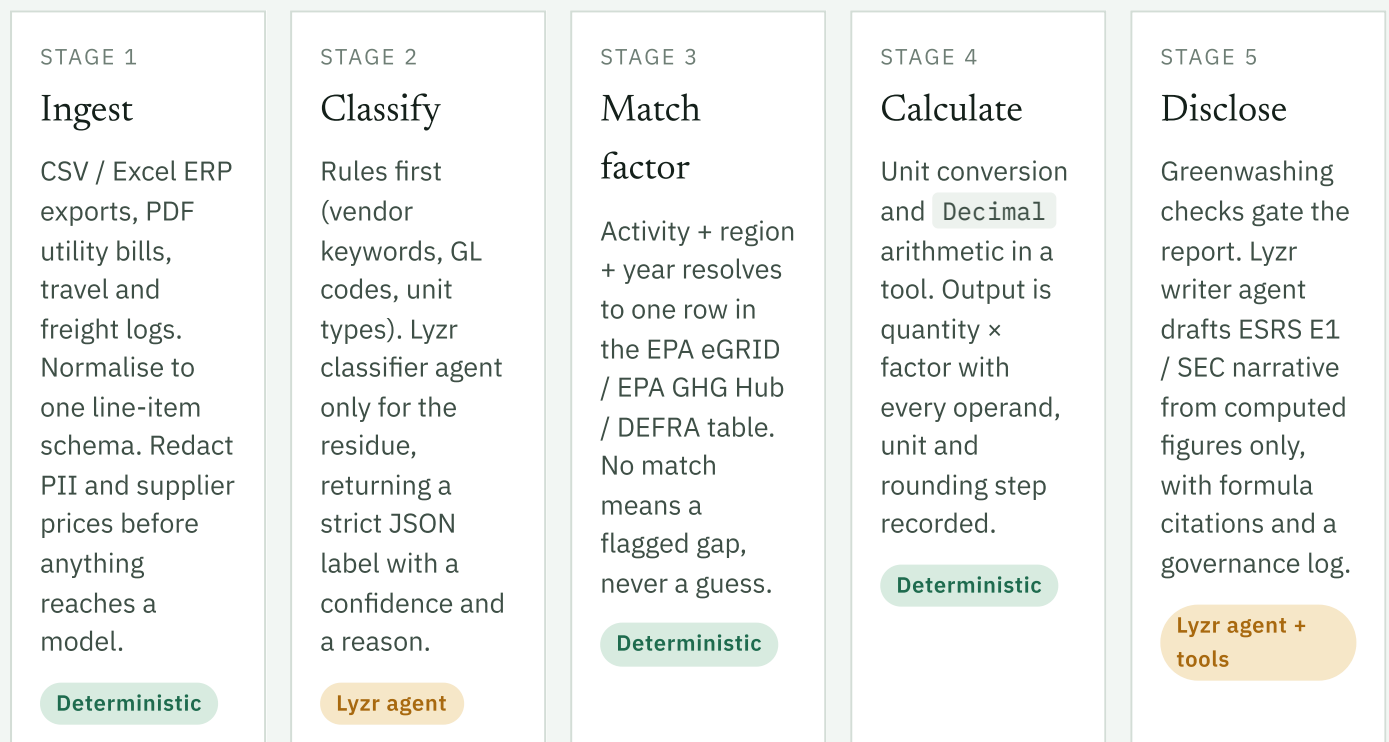
points

Sustainability dashboard UX

The approach in one sentence

The LLM is allowed to *read and label*. It is never allowed to *look up a number or multiply*. Everything numeric is a deterministic tool with a citation.

That split is the whole design. Lyzr agents handle the fuzzy work: reading a scanned utility bill, deciding that "Fleet fuel card, Shell, 412.6 L" is Scope 1 mobile combustion, drafting disclosure prose. Python tools handle everything with a unit on it: factor lookup against a bundled table with a version and a page reference, unit conversion, decimal multiplication, and the lineage record. The agent calls the tool; it does not answer in its place.



Why factor matching must be deterministic

The same activity has a different official factor depending on the jurisdiction, the year of the table, and the unit the invoice was written in. A model that "remembers" a factor will

pick one of these and present it with full confidence. The copilot instead ships a versioned factor table and cites the row it used.

ACTIVITY	SCOPE	SOURCE TABLE	FACTOR	UNIT
Grid electricity, US national average	2 (location-based)	EPA eGRID2022, US total output rate	0.3723	kg CO ₂ e / kWh
Grid electricity, UK	2 (location-based)	DEFRA 2024, UK electricity generation	0.20705	kg CO ₂ e / kWh
Natural gas, stationary combustion	1	EPA GHG Emission Factors Hub 2024, Table 1	53.06	kg CO ₂ / mmBtu
Diesel, average biofuel blend	1 (mobile)	DEFRA 2024, Fuels	2.51233	kg CO ₂ e / litre
Air travel, long-haul economy	3, Cat. 6	DEFRA 2024, Business travel, air (with RF)	0.14615	kg CO ₂ e / passenger·km
HGV freight, average laden	3, Cat. 4	DEFRA 2024, Freightng goods	0.10527	kg CO ₂ e / tonne·km

Illustrative rows from the tables the copilot will bundle. Each row carries a source, publication year, and page or sheet reference so the audit trail can cite it.

Greenwashing controls

These run as deterministic checks over the computed ledger before the disclosure agent is allowed to write anything. Each produces a finding with a severity, and blocking findings stop report generation.

Unsupported factor Blocks A line has a tCO ₂ e figure but no factor row reference. Should be impossible by construction; the check exists so the audit can prove it.	Scope 2 method drift Blocks Location-based and market-based figures mixed inside one total.
Unbacked claim Blocks Draft narrative contains "carbon neutral", "net zero" or "offset" without a matching offset ledger entry.	Implausible drop Warns Any scope falls more than 30% year-on-year with no activity-data drop to explain it.

Missing Scope 3 categories Warns

Purchased goods, upstream transport or business travel reported as zero while ERP shows spend in those GL codes.

Unit outlier Warns

A line's intensity sits more than 3σ from its category median, the usual sign of a kWh / MWh mix-up.

What we need to build it

From you

REQUIRED

A Lyzr Agent Studio account, an API key, and the agent IDs once the agents are created. Without these the backend runs in a local fallback mode that uses rules only, so the whole pipeline still demos.

OPTIONAL

Real or anonymised sample data: an ERP export, a couple of utility PDFs, a travel log. Otherwise we ship a realistic synthetic dataset for a fictional mid-size manufacturer.

DECISION

Primary jurisdiction for the demo report. Default is CSRD (ESRS E1) with an SEC variant as a second template.

Stack

- **Backend** Python 3.12, FastAPI, pandas, pdfplumber, decimal for all arithmetic, SQLite for the audit ledger.
- **Agents** Lyzr Agent API for classification and disclosure drafting; tool schemas defined in the repo so the calculation tools are callable from Lyzr.
- **Frontend** React + Vite dashboard: upload, ledger by scope, lineage drawer per line, findings panel, report export.
- **Data** Bundled EPA eGRID, EPA GHG Hub and DEFRA factor tables as versioned JSON with citations.
- **Run** Dockerfile and docker-compose for one-command startup.

Repository layout

```
AI-Quest-Beyond-the-Wrapper/  
├─ agents/                # Lyzr agent definitions, prompts, tool schemas, orchestra  
│   ├─ classifier/        # Scope 1/2/3 labeller (strict JSON out)  
│   ├─ disclosure_writer/  # CSRD / SEC narrative from computed figures only  
│   └─ tools/             # OpenAPI schemas Lyzr calls: match_factor, calculate, lin  
│       └─ orchestrator.py # Handoff between agents and deterministic tools  
├─ backend/               # FastAPI service  
│   ├─ ingestion/         # CSV, Excel, PDF parsers → line-item schema  
│   └─ factors/           # Versioned EPA / DEFRA tables + matcher
```

```
| | └─ calc/                # Units, Decimal engine, lineage records
| | └─ guardrails/         # PII redaction, greenwashing checks
| | └─ reports/            # ESRS E1 and SEC templates
| | └─ tests/              # Golden calculations the judges can re-run
└─ frontend/              # React + Vite dashboard
└─ Dockerfile
└─ docker-compose.yml
└─ .env.example
└─ README.md
```

Build order

Ordered so that the scoring-heavy parts land first and each phase is demoable on its own.

- 1 Calculation core** accuracy · 30 pts
Line-item schema, factor tables with citations, unit conversion, Decimal engine, lineage record. Golden tests against hand-computed figures.
- 2 Ingestion + guardrails** groundedness
CSV / Excel / PDF parsers, PII and price redaction, rule-based pre-classifier, greenwashing checks.
- 3 Lyzr agents + orchestration** architecture · 30 pts
Classifier and disclosure-writer agent configs, tool schemas, orchestrator with strict JSON contracts and a rules-only fallback when no key is present.
- 4 Audit ledger + reports** traceability · 20 pts
SQLite ledger, per-line lineage endpoint, ESRS E1 and SEC report renderers with formula citations and governance log.
- 5 Dashboard** UX · 20 pts
Upload, scope breakdown, category chart, lineage drawer, findings panel, report download.
- 6 Packaging** submission
Docker, compose, .env.example, README with architecture notes and a judge walkthrough.

How each rubric line is answered

PILLAR	WEIGHT	WHAT THE JUDGES WILL SEE
Lyzr architecture & tool calling	30	Two named agents with published configs, three deterministic tools with OpenAPI schemas, and an orchestrator log showing every handoff. No number ever originates inside a model response.
Emission calculation accuracy	30	Versioned factor tables with citations, explicit unit conversion table, Decimal arithmetic, golden test suite re-runnable with one command.
Auditability & traceability	20	Click any tCO ₂ e figure and see the source row, the redacted raw text, the factor row and its citation, the conversion, and the formula with operands.
Sustainability dashboard UX	20	CSO view: totals by scope and category, findings that need attention, report status. Auditor view: lineage drawer and governance log.

The six quest checkpoints map onto the same design. Hallucination mitigation and groundedness come from the tool boundary and citations. Retrieval quality is the factor matcher's exact-match-then-fallback strategy. Cost, latency and prompt architecture come from doing rules first and calling the agent only on the residue, with short strict-JSON prompts.